

MetroFlow

AI-Powered Metro Crowd Management & Scheduling Platform

Version 1.0
September 2026

Table of Contents

- 1. Project Overview
- 2. System Architecture
- 3. Modules Implemented
- 4. Technical Stack
- 5. How Each Module Was Implemented
- 6. API Endpoints
- 7. Database Schema
- 8. Deployment & Docker
- 9. Known Issues & Next Steps

1. Project Overview

MetroFlow is an AI-powered metro crowd management and scheduling platform that helps metro authorities monitor passenger flow, predict crowd density, and optimize train scheduling in real time.

Core Idea: Passengers make a prediction (station pair + time -> predicted occupancy), and the system automatically recommends schedule adjustments (headway, rake formation) based on that prediction.

Key Innovation: When you predict crowd density for a station pair, the system automatically filters and shows ALL trains on that corridor with updated schedule recommendations.

2. System Architecture

Frontend (React + Vite + Tailwind) -> FastAPI Backend (port 8000) -> SQLite Database

XGBoost Model -> Predictions -> Schedule Advisory (filtered by line/corridor)

3. Modules Implemented

Module 1: User Management - JWT auth with bcrypt, default admin account

Module 2: Crowd Monitoring - Real-time density prediction, alerts

Module 3: Scheduling Management - Prediction-driven schedule advisory

Module 4: AI Prediction - XGBoost model, $R^2=95.06\%$

Module 5: Alert & Notification - CRITICAL/MODERATE/NORMAL tiers

Module 6: Analytics Dashboard - KPIs, station performance

Module 7: Data Sources - 2023 dataset (need 2020-2027)

4. Technical Stack

Frontend: React 18 + Vite + Tailwind CSS (plain JS)

Backend: FastAPI + Uvicorn (Python 3.14)

ML: XGBoost Regressor (pre-trained JSON model)

Database: SQLite + SQLAlchemy (async)

Auth: JWT (jose) + bcrypt (HS256, 24hr expiry)

5. Key Implementation Detail

Prediction-to-Schedule Pipeline:

User selects Station A -> B, 18:00, Friday

-> handlePredict() -> POST /api/predict

-> Response: {predicted_occupancy: 2149, alert: CRITICAL}

-> setPredictionParams() + fetchScheduleAdvisory(params)

-> GET /api/schedule-advisory?from_station=...&to_station=...&line=...&hour=...

-> Backend filters dataset for matching trains

-> Response: {prediction_based: True, directives: [...]}

-> Frontend shows purple "AI-Powered" badge

6. API Endpoints

GET /health - Health check

GET /api/meta - Metro network metadata

POST /api/predict - Predict occupancy

GET /api/schedule-advisory - Schedule advisory (filtered)

GET /api/analytics - Analytics data

POST /api/auth/login - Login

POST /api/auth/register - Register

7. Database Schema

predictions table: id, from_station, to_station, line_color,
entry_hour, day_of_week, train_capacity, predicted_occupancy,
occupancy_rate_pct, recommended_headway, traffic_tier,
fleet_action, alert_level, created_at
users table: id, username, password_hash, role, is_active

8. Deployment

Docker: `docker-compose up -d --build`

Local: Backend on port 8000, Frontend on port 5173

Default Admin: admin / admin123

9. Known Issues

1. Dataset only covers 2023 (need 2020-2027)
2. Docker Desktop not installed/running
3. node_modules was missing (fixed with npm install)